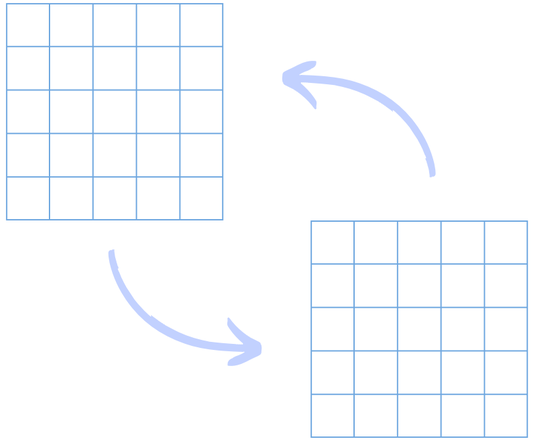
TIPOS DE REPOSITORIOS

para Base de Datos

1. BASES DE DATOS RELACIONALES
(RDBMS / SQL)

- Modelo más tradicional y utilizado. Organiza los datos en **tablas** con filas y columnas, conectadas entre sí mediante claves (llaves). Se basan en el principio **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad), lo que garantiza que las transacciones sean seguras.



Uso ideal

Sistemas financieros, inventarios, aplicaciones empresariales donde la integridad de los datos es crítica.

Nombres conocidos

PostgreSQL: El más avanzado y de código abierto.
MySQL: Muy popular para desarrollo web.
Oracle Database: Estándar en grandes corporaciones.
Microsoft SQL Server: Muy integrado en ecosistemas Windows.

2. BASES DE DATOS NOSQL
(NO SOLO SQL)

- Surgieron para **manejar grandes volúmenes de datos no estructurados** y permitir un escalado horizontal más sencillo. No utilizan el esquema rígido de tablas.
- Orientadas en:

Documentos

Almacenan la información en archivos similares a JSON o XML. Cada "documento" puede tener una estructura diferente.

Clave-Valor (Key-Value)

Es el **modelo más simple**: una clave única apunta a un valor específico. Son extremadamente rápidas.

Columnas (Wide-Column)

En lugar de leer filas enteras, **optimizan la lectura por columnas**. Son ideales para analítica masiva.

Basadas en Grafos

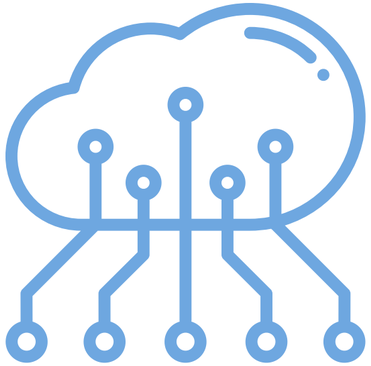
Se enfocan en las **relaciones** entre los datos. Utilizan "nodos" (entidades) y "aristas" (conexiones).

3. REPOSITORIOS DE ANÁLISIS Y ALMACENAMIENTO MASIVO (BIG DATA)

- A diferencia de las bases de datos anteriores (que son transaccionales u OLTP), estos están **diseñados para el análisis de datos históricos** (OLAP).

Data Warehouse (Almacén de Datos):

Repositorio central de datos estructurados provenientes de múltiples fuentes. Está optimizado para **consultas complejas y reportes.**



Data Lake (Lago de Datos):

Un depósito masivo que almacena datos en su formato original (crudos), ya sean **estructurados, semi-estructurados o no estructurados.**

4. NEW SQL

- Es un intento moderno de **combinar** lo mejor de ambos mundos: la escalabilidad horizontal de **NoSQL** con las garantías ACID y el lenguaje **SQL** de las relacionales.

SQL TRADICIONAL

Garantizaba datos perfectos (**Consistencia**), pero si el tráfico crecía demasiado, el servidor colapsaba porque no podía repartir la carga fácilmente.

NO SQL

Permitía crecer sin límites añadiendo servidores (**Escalabilidad**), pero a veces los datos tardaban unos segundos en actualizarse en todos lados, provocando errores en transacciones críticas.

NewSQL nace para romper esta elección, ofreciendo la escalabilidad de NoSQL sin sacrificar la seguridad de los datos de SQL.